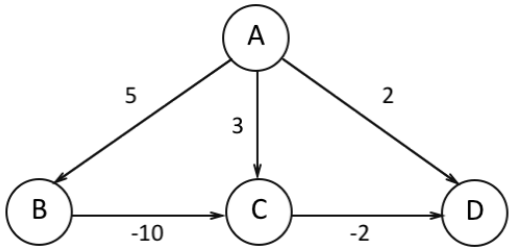
CSC4120 Design and Analysis of Algorithms, Fall 2026
CUHK-SZ — Due: Nov 18, 23:59
Costas Courcoubetis — Homework - Week 8

Problem 1. Dijkstra with negative weights (10 pts)

Consider the following directed graph with negative edges: $V = \{A, B, C, D\}$, $w_{AB} = 5, w_{AC} = 3, w_{AD} = 2, w_{BC} = -10, w_{CD} = -2$, starting node = A. Show the resulting iteration table (one row per iteration $0, 1, 2, ...$), with the value of the shortest distance for each node and the set of nodes in Q. What is the problem and why it happens?



Problem 2. (10 pts)

In the following modified Dijkstra's algorithm each vertex v will be pushed back to the priority queue Q each time it is successfully relaxed, i.e., $v.d > u.d + w(u, v)$, unless we are certain that it was there already, i.e., $v.d = \infty$. This guarantees that each time a vertex v is found to have a shorter path than before, the algorithm will re-examine the shortest paths through its neighbours (this will happen when v will be eventually enter S and this can happen even if it was in S previously). One can check that this algorithm can solve the single-source shortest path in a graph with negative weighted edges as long as there are no negative cycles.

```
DIJKSTRA (G,w,s)

1    INITIALIZE-SINGLE-SOURCE (G,s)
2    S = ∅
3    Q = G.V
4    while Q ≠ ∅
5        u = EXTRACT-MIN (Q)
6        S = S ∪ {u}
7        for each vertex v ∈ G.Adj[u]
8            RELAX (u,v,w)
```

```
RELAX (u,v,w)

1    if v.d > u.d + w(u,v)
2        v.d = u.d + w(u,v)
3        v.π = u
```

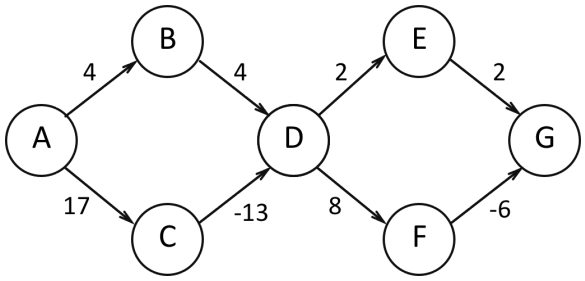
```
MODIFIED_DIJKSTRA (G,w,s)

1    INITIALIZE-SINGLE-SOURCE (G,s)
2    S = ∅
3    Q = G.V
4    while Q ≠ ∅
5        u = EXTRACT-MIN (Q)
6        S = S ∪ {u}
7        for each vertex v ∈ G.Adj[u]
8            MODIFIED_RELAX (Q,u,v,w)
```

```
MODIFIED_RELAX (Q,u,v,w)

1    if v.d > u.d + w(u,v)
2        if v.d ≠ ∞ :
3            push(Q,v)
4        v.d = u.d + w(u,v)
5        v.π = u
```

Use the modified Dijkstra's algorithm to answer the following questions:
In this scenario, traversal order refers to the order of nodes extracted by EXTRACT-MIN(Q).

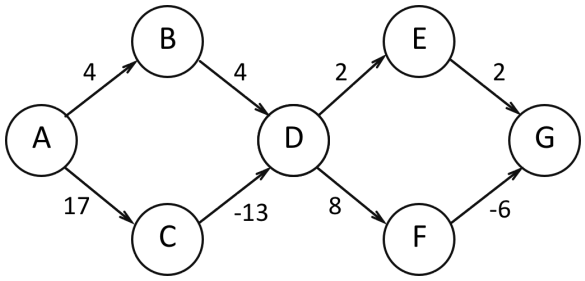


(a) List the traversal order of the first seven vertices, starting from A.

(b) List the next six.

Problem 3. (10 pts)

Use Bellman-Ford algorithm to answer the following questions for the following graph.



Show the iteration table and resulting shortest-path tree, assuming the order of edges relaxed at each iteration = A-B, B-D, A-C, C-D, D-E, E-G, D-F, F-G.

Problem 4. (15 pts)

Consider a graph G in which its vertices V represent cities, and the edges E represent roads that connect cities. A delivery company is planning a route from a particular city u to another city v. This company knows the probability $p(e) \in [0, 1]$ that a given road $e \in E$ is closed (blocked). A route connecting two cities is a sequence of roads and it is closed if any of its roads is closed. Alternatively, a route is open if all its roads are open.

(a) Use Dijkstra's algorithm to find a route with minimum probability of being closed. Assume that road closures are independent.

Hint: Transform the problem so that the optimization problem becomes one of finding shortest paths for appropriately defined positive weights on the edges.

(b) Make appropriate changes to Dijkstra's algorithm to compute paths with the largest product of edge weights and use this algorithm to solve the problem.

Problem 5. (15 pts)

Consider a directed graph in which the only edges with negative weights are those that leave the initial node s and all other edges are positive. Can Dijkstra's algorithm, started at s, fail on such a graph? Prove your answer.

Problem 6. (20 pts)

You are given a directed graph with (possibly negative) weighted edges, in which the shortest path between any two vertices is guaranteed to have at most k edges. Give an algorithm that finds the shortest path between two vertices u and v in $O(k|E|)$ time.

Problem 7. (20 pts)

We consider the case of speeding up the search for the shortest path to destination t using modified edge weight based on the potential function method.
Prove that the potential computed using landmarks is always consistent (same as 'admissible', 'feasible').
In particular, prove that property for the case of using a single landmark and then for the case of using multiple landmarks.

Problem 8. (Optional, 30 pts)

Problem 4.7 in DPV. This is a harder problem, solve it for a bonus of 30 points.
(Maximum points: 130/100.) I suggest that you definitely try it by using the next hint.
Problem: You are given a directed graph $G = (V, E)$ with (possibly negative) weighted edges, along with a specific node $s \in V$ and a tree $T = (V, E'), E' \subseteq E$. Give an algorithm that checks whether T is a shortest-path tree for G with starting point s. Your algorithm should run in linear time.
Hint: Let $d_T(u, v)$ be the distance between u and v along the edges of T. Explain how it can be computed it in linear time. Then, consider the following test: for any edge $(u, v) = e \in E - E'$, i.e., every e not belonging to the tree, check that $w(u, v) + d_T(s, u) \geq d_T(s, v)$. Prove that this will succeed for all $e \in E - E'$ if and only if T is a correct shortest path tree from s. How to prove that? Consider a run of the Bellman-Ford algorithm for shortest paths starting at s. Consider the first update update (u, v) causing the distance of v to drop below $d_T(s, v)$. Show that this edge (u, v) will not pass the test defined above.